A Personalized Video Game Recommender

Shiny App: https://gamematch-shiny-127112588159.us-central1.run.app

Flask API: https://gamematch-api-127112588159.us-central1.run.app

Cindy Shih

Stats 418 Final Project

2026.06.02

Project Overview & Motivation

Problem

Players often have limited time, but there are many games to choose from.

Traditional recommenders usually emphasize rating, popularity, or genre. Those signals do not always answer: “What do I want to play right now?”

My Solution

The app turns a player’s current situation into a ranked list of recommendations.

The app asks for mood, platform, solo/multiplayer preference, and session length, then returns match scores that are easy to compare.

Context-aware recommendations

- Play vibe
- Platform
- Play style
- Session length

Data Collection & Preprocessing

RAWG API
HTTP requests
~500 games

→

Raw CSV
game metadata

→

Preprocessing
Clean + Engineer

Features
for scoring

Collected Fields

- Title
- Genres and tags
- Platforms
- Rating and rating count
- Playtime
- Image URL

Cleaning

- Parsed nested list columns
- Removed duplicates
- Standardized platforms into PC, PlayStation, Xbox, and Nintendo Switch
- Treated zero playtime as unknown

Feature Engineering

- Assigned one primary play vibe
- Allowed multi-label play style
- Built session length from playtime, genres, and tags

Model Development & Evaluation

Recommendation Model

Weighted scoring model

Play vibe	45%
Play style	20%
Session length	15%
RAWG rating	15%
Rating count	5%

Platform is a hard filter: unavailable games are removed before ranking.

Evaluation Approach

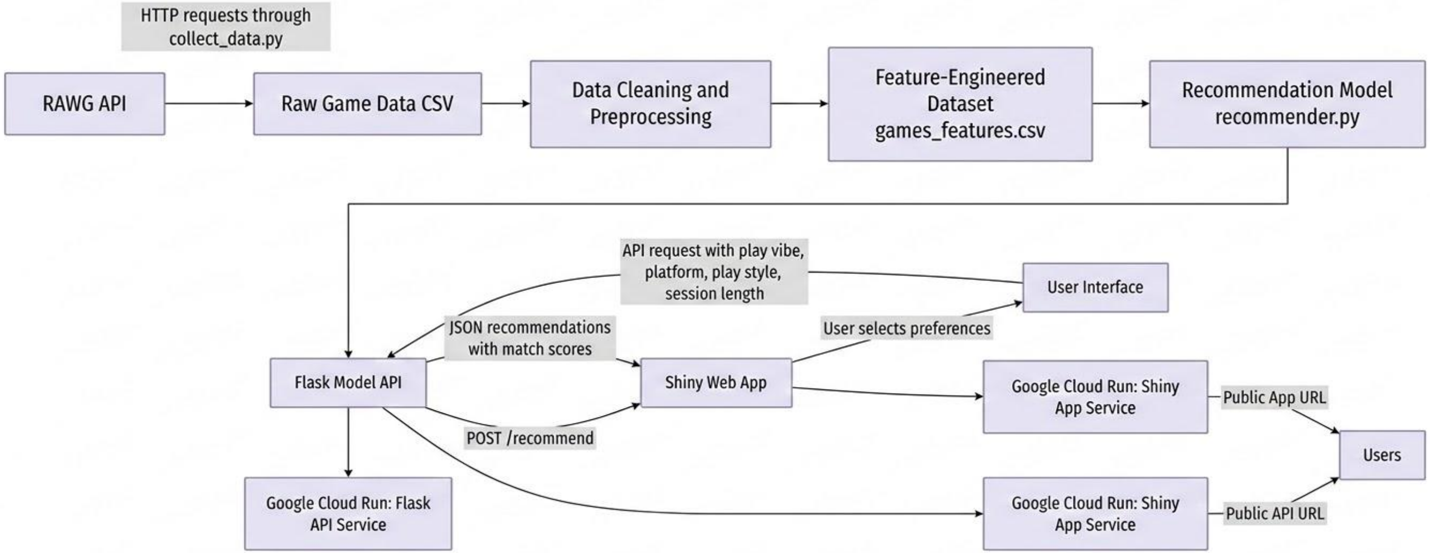
Manual testing approach

- Tested many input combinations (vibe × platform × style × session length)
- Verfied whether results matched selected filter criteria
- Reviewed ranking consistency across repeated queries

Match score = interpretability measure, not accuracy.

Final Solution Architecture Diagram

User selects preferences in Shiny → app calls Flask API → API returns ranked games with match scores.



Application Features & Walkthrough

User Flow

Choose your preferences

Play vibe

Chill / Cozy

Platform

PC

Play style

Solo

Session length

Short Session (≤ 1 Hour

Recommendations

3 5 10

Find games

1 **Choose preferences**

play vibe, platform, play style, session length, no. of games (3-10)

2 **Click Find games**

Shiny sends the inputs to the Flask API

3 **View ranked results**

recommendations show match score, title, rating and image

- Chill / Cozy
- Competitive / Intense
- ✓ Story-Driven
- Puzzle / Strategy
- Creative / Exploration

- ✓ PC
- PlayStation
- Xbox
- Nintendo Switch

- ✓ Solo
- Multiplayer / Co-op

- ✓ Short Session (≤ 1 Hour)
- Medium Session (1–3 Hours)
- Long Session (3+ Hours)

Example output: Chill / Cozy + PC + Solo + Short Session

Found 5 recommendations

Using Chill / Cozy, PC, Solo, Short Session (≤ 1 Hour).

95.7% match

Sonic 3 and Knuckles

Vibe: Chill / Cozy | Platform: PC | Style: Solo, Multiplayer / Co-op | Session: Short Session (≤ 1 Hour)

Rating: 4.41

95.1% match

HoloCure - Save the Fans!

Vibe: Chill / Cozy | Platform: PC | Style: Solo | Session: Short Session (≤ 1 Hour)

Challenges, Lessons & Future Work

Challenges → Solutions

• Missing playtime values → Treated zero as unknown and Session length was built from genres, tags & playtime.

• No direct “play vibe” field → Created custom mappings from genres and tags

• Scores inflated too easily → Platform became a hard filter and each game received one primary vibe

Lessons Learned

• Simple models can be useful when the logic is transparent

• Feature engineering mattered more than model complexity

• AI helped speed up coding, but testing and final decisions still required human judgment

Future Improvements

• Add user feedback buttons: like / not interested

• Expand the dataset beyond the initial RAWG collection

• Add skill level, price, and budget filters

• Use real user testing for stronger evaluation